

NutriAgent AI

מסמך אפיון טכני לפיתוח ב-Cursor

מסמך זה מהווה אפיון טכני מלא, המבוסס על מסמך דרישות הלקוח (BRD) ומיועד לשמש כבסיס עבודה ישיר עבור פיתוח באמצעות Cursor / סוכן קידוד AI. המסמך כולל את שם הפרויקט, מטרתו, סוגי המשתמשים, המודלים, הארכיטקטורה, הטכנולוגיות, המסכים, מבנה מסד הנתונים (ERD), דרישות סביבתיות ל-GCP, תרחישי שימוש (User Stories) ומנגנון תיעוד לוגים (Audit Log) עם שכבת Caching למנהל המערכת.

1. שם הפרויקט, מטרה ופונקציונליות

שם הפרויקט: NutriAgent AI

מטרת הפרויקט:

אפליקציה המסייעת למשתמשים לנהל ולעקוב אחר התזונה היומית שלהם באופן אוטומטי ופשוט. המשתמש מצלם ארוחה בתוך האפליקציה, והמערכת מזהה באמצעות מודל ראייה ממוחשבת (Vision) את סוגי המזון והכמויות המוערכות בצלחת, מחשבת ערכים תזונתיים, ומספקת בצ'אט תשובה עם המלצות המשך מותאמות אישית ליעדי הדיאטה.

פונקציונליות עיקרית:

- זיהוי אוטומטי של מזון מתוך תמונה (Vision Agent) וחישוב ערכים תזונתיים (קלוריות, חלבון, שומן, פחמימות, סוכר).
- שמירת נתוני הארוחות במסד נתונים לצורך ניתוח היסטורי, גרפים ודוחות.
- צ'אט חכם (Multi-Agent) למתן המלצות תזונה מותאמות אישית, כולל שאילתות חופשיות בשפה טבעית.
- דשבורד גרפי אישי המציג פילוח תזונתי יומי/שבועי/חודשי.
- ניהול פרופיל אישי: יעדי דיאטה, הגבלות בריאותיות, אלרגיות, סוג תזונה.
- פאנל ניהול (Admin) לניהול משתמשים, הרשאות, וצפייה בלוגים ברמת המערכת.
- תיעוד ושקיפות ("Clinical Glass Box") - כל המלצה מתועדת יחד עם המקורות עליהם התבססה.

2. משתמשי האפליקציה

הכניסה למערכת מתבצעת באמצעות שם משתמש וסיסמה אישיים (עם אפשרות עתידית ל-MFA). קיימים שני סוגי תפקידים עיקריים, המנוהלים דרך טבלת ניהול משתמשים (Roles):

מה יכול לעשות	מי זה	תפקיד
צילום ארוחות · צפייה בגרפים אישיים · שיחה עם הצ'אט וקבלת המלצות · עריכת פרופיל אישי ויעדי תזונה	אדם פרטי ללא ידע טכני, המנהל את הדיאטה האישית שלו	User (משתמש קצה)
כל הרשאות ה-User, בנוסף: ניהול טבלת המשתמשים · קביעת הרשאות (User/Admin) · צפייה בנתוני מערכת מלאים · צפייה ומעקב אחר לוגים (Audit Log) בפאנל ניהול ייעודי	צוות ניהול/תפעול המערכת	Admin (מנהל מערכת)

3. מודלים באפליקציה (AI Agents / Models)

המערכת בנויה כארכיטקטורת Multi-Agent, כאשר כל סוכן פועל כשירות עצמאי (Microservice) המחובר לעמוד שדרה משותף (MCP - Model Context Protocol):

סוכן / מודל	תפקיד
Router / Orchestrator	מבוסס LangGraph; מסווג את כוונת הפנייה (Intent Classification) ומנתב לסוכן המתאים
Vision Agent	מודל ראייה ממוחשבת לזיהוי סוגי מזון, פילוח רכיבים ואומדן משקל/מנה מתוך תמונה
Nutrition Agent	מודל שפה (LLM) המחשב ערכים תזונתיים (מאקרו ומיקרו) ובודק התאמה מול הגבלות/אלרגיות המשתמש ומול נורמות תזונתיות (DRI/WHO)
RAG Agent + Reranker	שליפת מידע תזונתי ממאגרי ידע פנימיים/חיצוניים ודירוגו לפי רלוונטיות, לפני העברה למודל השפה
Text2SQL Agent	המרת שאלות בשפה טבעית על ההיסטוריה האישית לשאילתות מובנות על מסד הנתונים
GraphDB Agent	שכבת ידע לקשרים סיבתיים (הגבלה בריאותית ↔ תחליף מזון בטוח), להמלצות מדויקות ובטוחות יותר
Model Routing	ניתוב גמיש בין ספקי מודלים (למשל GPT-4o למשימות חזותיות, Claude למשימות ניתוח/הסקה) לפי עלות וביצועים

4. ארכיטקטורת האפליקציה

הארכיטקטורה מחולקת לשכבות (Layers) הבאות:

- Client Layer - אפליקציית מובייל מבוססת React Native; פורטל ניהול נפרד (Admin Portal) מבוסס Next.js.
 - API / Gateway Layer - שער כניסה מאובטח (Auth, Rate Limiting) המפנה בקשות ל-Orchestrator.
 - Router - Orchestration Layer מבוסס LangGraph המנתב בין הסוכנים המומחים (Agentic Flow) על גבי MCP.
 - Agents Layer (Microservices) - Vision Agent, Nutrition Agent, RAG Agent + Reranker, Text2SQL Agent, GraphDB Agent - כל אחד שירות עצמאי ב-Cloud Run.
 - Data Layer - Postgres + pgVector כמאגר מרכזי (רלציוני + וקטורי); אפשרות הרחבה עתידית ל-Neo4j/GraphDB (ידע קליני), Redis (זיכרון שיחה קצר-טווח ו-RAG), Weaviate (Caching בקנה מידה גדול).
 - Cross-Cutting: Security (Auth/Authorization, הצפנה At Rest/In Transit), Logging & Audit (AuditLog + Redis), Monitoring/SOC-SIEM (Cache).
- זרימת עבודה טיפוסית: Client → API Gateway → Orchestrator (Intent Classification) → סוכן מתאים (Vision / RAG / Nutrition Agent → Text2SQL / GraphDB) (סינתזה) → תשובה בצ'אט + שמירה ב-DB + רישום ב-AuditLog.

4.1 ניווט ראשי - מבנה Tabs

שכבת ה-Client (הן במובייל והן בדסקטופ) נבנית סביב שישה טאבים קבועים, המהווים את הניווט הראשי של האפליקציה. כל טאב הוא מסך/View עצמאי הצורך את אותו ה-API, כך שהלוגיקה העסקית משותפת בין הפלטפורמות ורק שכבת התצוגה משתנה:

#	טאב	תוכן מרכזי
1	Chat (ראשי)	מסך הבית - העלאת תמונת אוכל/צילום ארוחה, שיחה חופשית עם הסוכן, קבלת ניתוח ותשובות בזמן אמת
2	Dashboard	גרפים מרכזיים: Calorie Budget (יעד מול צריכה בפועל), פילוח Breakfast / Lunch / Dinner, מעקב צעדים (Steps)
3	Summary & Foods	רשימת ארוחות/מזונות שנרשמו בטווח זמן נבחר (יום/שבוע/חודש), עם אפשרות סינון וחיפוש
4	Meal Analysis	פירוט של ארוחה בודדת שנבחרה - תמונה, פריטי המזון שזוהו ורמת הוודאות של הזיהוי

#	טאב	תוכן מרכזי
5	Calories & Nutrients	פירוט תזונתי מלא: קלוריות, חלבון, שומן, פחמימות, סוכר וערכי מיקרו, ברמת ארוחה/יום/תקופה
6	Settings	הגדרות פרופיל תזונתי: יעדי דיאטה, הגבלות בריאותיות, אלרגיות וסוג תזונה - משמש כבסיס לסינון ולהתאמה אישית של המלצות הסוכן בכל האפליקציה

דרישת-על על (Non-Functional) - Responsive Design: האפליקציה חייבת לתמוך באופן מלא הן בדסקטופ (Web/Admin Portal) והן במובייל (React Native), עם אותו מבנה טאבים בשני הפלטפורמות:

- במובייל: הטאבים מוצגים כ-Bottom Tab Bar קבוע (אייקון + טקסט קצר), בהתאם למוסכמות iOS/Android.
- בדסקטופ/Web: אותם שישה טאבים מוצגים כ-Sidebar צדדי קבוע, כאשר תוכן כל טאב נפרש ברוחב מלא ומנצל את שטח המסך הנוסף (למשל Grid רב-טורי בדשבורד).
- יש לבנות את שכבת ה-UI לפי עקרון Mobile-First עם Breakpoints (למשל עד 768px = מובייל, מעל = דסקטופ), כך שרכיב אחד (קומפוננטה) יתאים את עצמו לשני המצבים ללא כפילות קוד עסקי.

4.2 תרחישי זרימה טכניים (Use Case Flows)

להלן שלושה תרחישי זרימה מרכזיים, המתארים כיצד הטאבים, הסוכנים (Agents) ומסד הנתונים פועלים יחד בפועל:

1 Use Case - זיהוי ארוחה מצילום ועדכון אוטומטי של הנתונים:

- המשתמש נכנס לטאב Chat ולוחץ על כפתור המצלמה בתוך האפליקציה; המערכת מבקשת אישור הרשאת מצלמה.
- המשתמש מצלם את המזון שהוא מתכנן לאכול ולוחץ "אישור העלאה" - התמונה עולה לצ'אט.
- ה-Vision Agent "רואה" את התמונה ומפענח את תוכן הצלחת.
- הפנייה מנותבת ל-Router/Orchestrator, המפענח את גודל המנה ומחלק את הצלחת לסוגי מזון (Food Segmentation) - כלומר מזהה אילו סוגי מזון קיימים ומחלק כל אחד למרכיביו.
- ה-Nutrition Agent מחשב ערכים תזונתיים (קלוריות, חלבון, שומן, פחמימות וכו') לכל מרכיב שזוהה.
- התוצאה נשמרת ב-DB (Meals, MealItems, NutritionValues) בשיוך למשתמש, לתאריך ולשעה.
- המשתמש מקבל בצ'אט תצוגה גרפית (Nutrient Card / גרף עוגה) של כמויות וסוגי המזון והערכים התזונתיים שזוהו.
- הנתונים מתעדכנים אוטומטית וללא פעולה נוספת בטאב Dashboard, בגרפים של אחוזי התזונה היומיים.
- בנוסף לגרף, הסוכן מוסיף בצ'אט סיכום טקסטואלי קצר וברור: סה"כ קלוריות, חלבון, שומן, פחמימות שנאכלו בארוחה זו.

2 Use Case - המלצות תזונה מותאמות אישית:

- המשתמש שואל בצ'אט שאלה חופשית, למשל "מה כדאי לי לאכול עכשיו?".
- הפנייה מנותבת דרך ה-Router ל-RAG Agent + Reranker.
- ה-RAG Agent שולף המלצות ממאגר/פורטל תזונה בין-לאומי ואמין, ומצרף לתשובה את המקורות עליהם התבססה (שקיפות - Clinical Glass Box).
- ההמלצה מסוננת ומותאמת מול יעדי הדיאטה, ההגבלות הבריאותיות והאלרגיות של המשתמש - כפי שהוגדרו מראש בטאב (Settings) (UserProfile).
- הסוכן מחזיר בצ'אט המלצה מנומקת, ברורה ובטוחה עבור המשתמש הספציפי, כולל ציון המקור.

3 Use Case - שאילתה על היסטוריה בשפה טבעית (Text2SQL):

- המשתמש שואל בצ'אט, למשל "מה אכלתי אתמול?".
- הפנייה מנותבת דרך ה-Router ל-Text2SQL Agent.
- ה-Text2SQL Agent שולח שאילתה מובנית ל-DB (טבלאות Meals / MealItems / NutritionValues), מסוננת לפי מזהה המשתמש וטווח התאריכים הרלוונטי.
- תוצאת השאילתה מתורגמת חזרה לשפה טבעית וברורה, ומוצגת למשתמש בצ'אט כתשובה מסכמת.

5. טכנולוגיות בכל מודל

מודול	טכנולוגיות מרכזיות
Frontend - מובייל	React Native, Expo, TypeScript
Frontend - Admin Portal	Next.js, TypeScript, React Query
Backend / API	Python (FastAPI), REST/GraphQL, JWT Auth או Node.js
Orchestration	LangChain (ניתוב וזרימת סוכנים), LangGraph (ניהול שיחה וכלים)
Vision	מודל Vision (למשל GPT-4o Vision / מודל ייעודי לזיהוי מזון)
LLM / Reasoning	Model Routing לפי Claude / GPT-4o
RAG	RAG + Reranker על גבי מאגר וקטורי (pgVector, בהמשך Weaviate)
Text2SQL	שכבת המרת שפה טבעית ל-SQL על Postgres
GraphDB	Neo4j (עתידי) - ידע קליני על הגבלות ותחליפי מזון
Database	Redis, Neo4j, Weaviate; PostgreSQL + pgVector (POC); הרחבה עתידית:
Caching	Redis - Cache ללוגים אקטואליים (Admin) וזיכרון שיחה קצר-טווח
Infra / Hosting	Google Cloud Platform (GCP): Cloud Run (Serverless Backend), Cloud POC לשלב SQL/Postgres, Free Tier
Security	Authentication (Username/Password עתידי), MFA, Authorization מבוסס Roles, הצפנה At Rest ו-In Transit

6. מסכים עיקריים

עקרונות מנחה לעיצוב: פעולה מינימלית ("צילום ← תוצאה"), שפה פשוטה ולא טכנית, משוב חזותי מיידי ונגישות מלאה. המסכים הבאים בונים את חמשת הטאבים המוגדרים בסעיף 4.1, בתוספת מסכי כניסה/הגדרות וניהול:

- מסך התחברות / הרשמה (Login / Sign-Up) עם שם משתמש וסיסמה.
 - שאלון קליטה (Onboarding) - הזנת יעדי דיאטה, הגבלות בריאותיות, אלרגיות וסוג תזונה.
- טאב 1 - Chat (ראשי):**
- כפתור צילום/העלאת תמונת ארוחה בולט במרכז המסך.
 - חלון צ'אט מתמשך - שאלות המשך, בקשות ייעוץ ותשובות מהסוכן.
 - כרטיס תוצאה (Nutrient Card) המוצג בצ'אט מיד לאחר הזיהוי - ערכים תזונתיים בצורה ויזואלית (progress bars / אחוזים).
- טאב 2 - Dashboard:**
- גרף Calorie Budget - עיגול/פס התקדמות המראה יעד יומי מול צריכה בפועל.
 - פילוח Breakfast / Lunch / Dinner - כרטיסים או עמודות המציגים קלוריות לכל ארוחה בציר היום.
 - מד Steps - מעקב צעדים יומי (אינטגרציה עתידית עם חיישן/אפליקציית בריאות).
- טאב 3 - Summary & Foods:**
- רשימת ארוחות/מזונות לפי טווח תאריכים נבחר (יום/שבוע/חודש), עם חיפוש וסינון.
 - כל שורה ברשימה מובילה למסך Meal Analysis של אותה ארוחה.
- טאב 4 - Meal Analysis:**
- תצוגת התמונה שצולמה לצד רשימת פריטי המזון שזוהו ורמת הוודאות (Vision Confidence) לכל פריט.
 - אפשרות לתקן/להשלים ידנית פריט מזון שזוהה בטעות.
- טאב 5 - Calories & Nutrients:**
- פירוט תזונתי מלא ברמת ארוחה/יום/תקופה: קלוריות, חלבון, שומן, פחמימות, סוכר.

תצוגה גרפית משולבת (עוגה / עמודות / קו) לפי בחירת המשתמש בטווח הזמן.

טאב 6 - Settings:

- עריכת יעדי דיאטה (למשל יעד קלוריות/חלבון יומי).
- הגדרת הגבלות בריאותיות ואלרגיות (למשל רגישות לגלוטן, אגוזים).
- בחירת סוג תזונה (קטוגנית / צמחונית / דלת פחמימות וכו').
- הנתונים בטאב זה נשמרים ב-UserProfile ומשמשים את כלל הסוכנים (Nutrition Agent, RAG Agent) לסינון והתאמה אישית של המלצות בכל האפליקציה.

מסכי הגדרות וניהול נוספים:

- מסך פרופיל אישי - עריכת יעדי דיאטה, הגבלות ואלרגיות.
- ממשק ניהול (Admin Panel) - ניהול משתמשים והרשאות, נפרד מממשק המשתמש הרגיל.
- מסך לוגים (Admin - Audit Log Viewer) - צפייה בלוגים אקטואליים (מוזרמים מ-Cache) וחיפוש/סינון בהיסטוריה המלאה.

7. ERD ומסד נתונים (DB)

הקשרים המרכזיים בין הישויות:

- Users (N) → Roles (1) - כל משתמש משויך לתפקיד אחד (User / Admin).
- Users (1) → UserProfile (1) - לכל משתמש פרופיל תזונתי/בריאותי אחד.
- Users (1) → Meals (N) - משתמש יכול לתעד ריבוי ארוחות.
- Meals (1) → MealItems (N) - בכל ארוחה עשויים להיות מספר פריטי מזון.
- MealItems (1) → NutritionValues (1) - לכל פריט מזון מחושבים ערכים תזונתיים.
- Users (1) → ChatHistory (N) - עם שיוך אופציונלי בין הודעת צ'אט לארוחה רלוונטית.
- Users (1) → AuditLog (N) - לצורך מעקב ובקרת אבטחה מול SOC/SIEM.

ישות (Entity)	שדות מרכזיים
Roles	role_id (PK), role_name (User/Admin)
Users	,user_id (PK), name, email, password_hash, role_id (FK), created_at, account_status
UserProfile	,profile_id (PK), user_id (FK), diet_goals, health_restrictions, allergies, diet_type, weight, height, age
Meals	,meal_id (PK), user_id (FK), meal_datetime, source (home/restaurant), image_url
MealItems	,item_id (PK), meal_id (FK), food_type, estimated_quantity, vision_confidence
NutritionValues	value_id (PK), item_id (FK), calories, protein, fat, carbs, sugar
ChatHistory	,message_id (PK), user_id (FK), meal_id (FK, nullable), content, timestamp
AuditLog	log_id (PK), user_id (FK), action_type, timestamp, source_ip

הערות ל-DBA / מפתח:

- יש לוודא נרמול מתאים (Normalization) ומפתחות זרים (Foreign Keys) תקינים בין כל הישויות.
- יש לבחון יחסי Many-to-Many במידת הצורך (למשל בין משתמשים למגבלות בריאותיות מרובות - טבלת צומת ייעודית).
- יש להוסיף שדות ביקורת (created_at / updated_at) בכל טבלה מרכזית.

- טבלת AuditLog נכתבת ב-Postgres כמקור אמת מלא (Source of Truth); שכבת Redis משמשת אך ורק כ-Cache לרשומות האקטואליות (ראה סעיף 10).

8. דרישות למפתח / הנחיות ל-Cursor

- סביבת אירוח (Hosting):
- פריסה בענן - Google Cloud Platform (GCP).
- שימוש בשכבת Free Tier לצורך שלב הפיתוח / הוכחת היתכנות (POC).
- Cloud Run - להרצת שירותי הבק-אנד (Backend) כ-Serverless, כל סוכן כ-Service נפרד.
- Postgres + pgVector (למשל Cloud SQL) - אחסון רלציוני + Vector Store לשכבת RAG, במסגרת ה-Free Tier לשלב POC.
- Redis (Memorystore), שכבת Caching - (Free/Minimal) ללוגים אקטואליים וזיכרון שיחה קצר-טווח.
- הנחיות מבנה פרויקט מומלץ (Monorepo):
- apps/mobile/ - קוד React Native
- apps/admin-portal/ - קוד Next.js
- services/orchestrator/ - שירות Router/LangGraph
- services/text2sql-agent/, services/rag-agent/, services/nutrition-agent/, services/vision-agent, services/graphdb-agent/ - כל אחד שירות Cloud Run עצמאי
- infra/ - קבצי Terraform/IaC להגדרת GCP (Cloud Run, Cloud SQL, Memorystore, IAM)
- db/ - סכמות SQL, מיגרציות (למשל Prisma Migrate / Alembic) ותרשימי ERD
- דרישות טכניות נוספות:
- כל שירות חייב Dockerfile ותצורת deploy ל-Cloud Run (gcloud run deploy).
- משתני סביבה (Secrets) יאוחסנו ב-Secret Manager של GCP ולא בקוד.
- Authentication מבוסס JWT; Authorization מבוסס Roles (User/Admin) בבדיקת Middleware בכל endpoint.
- הצפנת נתונים רגישים At Rest (הצפנת DB) ו-HTTPS/TLS (In Transit) בכל התקשורת).
- כל קריאה לסוכן AI תתועד ב-AuditLog (כולל שמות המקורות עליהם התבססה ההמלצה - Clinical Glass Box).
- יש לכתוב טסטים אוטומטיים בסיסיים (Unit/Integration) לכל שירות לפני מיזוג ל-main.

9. User Stories עיקריים

#	תרחיש	משתמש	תיאור זרימה	תוצאה
1	זיהוי ארוחה ושמירת נתונים	User	המשתמש מצלם ארוחה → Vision Agent מזהה מזון וכמות → RAG+Reranker שולף ערכים תזונתיים → נשמר ב-DB בשיוך למשתמש/תאריך	המשתמש מקבל בציאט אישור ופירוט הערכים התזונתיים של הארוחה
2	הצגה ויזואלית של נתונים	User	המשתמש בוחר תאריך/טווח → המערכת שולפת ארוחות מה-DB → מציגה פילוח גרפי (חלבון/שומן/פחמימות/קלוריות)	דשבורד ברור וקריא המציג את מצב התזונה היומי/תקופתי
3	ייעוץ לבחירת מנה במסעדה	User	LangGraph Agent שולף פרופיל+הגבלות RAG+Reranker → Text2SQL מאתר תפריט/מידע רלוונטי → מייצר המלצה מנומקת	המשתמש מקבל בציאט המלצה ברורה למנה המתאימה, עם הסבר תזונתי קצר
4	ניהול משתמשים והרשאות	Admin	מנהל נכנס לפאנל הניהול → צופה בטבלת משתמשים → משנה תפקיד (User/Admin) או סטטוס חשבון	עדכון הרשאות המשתמש משתקף מיידית במערכת

#	תרחיש	משתמש	תיאור זרימה	תוצאה
5	צפייה במעקב לוגים (Audit)	Admin	מנהל נכנס למסך הלוגים → המערכת שולפת לוגים אקטואליים מ-Redis Cache ולוגים היסטוריים מ-Postgres	מנהל רואה תיעוד מלא ומעודכן של פעולות אבטחה/מערכת, לצורך בקרה מול SOC/SIEM

10. שמירת לוגים ותיעוד - גישת Admin עם שכבת Caching

דרישה: מנהל המערכת (Admin) חייב יכולת צפייה מלאה בלוגי המערכת (Audit Log) לצורכי בקרה, אבטחה ותאימות רגולטורית. על מנת לספק חוויית צפייה מהירה בלוגים ה"אקטואליים" (הפעילות האחרונה בזמן אמת), תשמש שכבת Caching ייעודית (Redis) - ואילו כלל היסטוריית הלוגים תישמר במקביל במסד הנתונים הרלציוני (Postgres) כמקור אמת מלא וארוך טווח.

ארכיטקטורת הלוגים:

- מקור אמת (Source of Truth): טבלת AuditLog ב-Postgres - כל אירוע (התחברות, שינוי הרשאה, קריאה לסוכן AI, שגיאת מערכת) נכתב אליה באופן סינכרוני ובלתי הפיך (Append-Only).
 - שכבת Cache: Redis - בכל כתיבת לוג חדש, הרשומה נכתבת גם ל-Redis (למשל כ-List/Sorted Set לפי timestamp), כדי לאפשר שליפה מהירה של הלוגים האקטואליים (למשל 24-48 שעות אחרונות / N הרשומות האחרונות) ללא צורך בשאילתת DB כבדה.
 - TTL - Time To Live: רשומות ב-Redis יוגדרו עם תפוגה אוטומטית (למשל 48 שעות), כך ש-Cache יישאר "רזה" ומכיל רק את הפעילות העדכנית ביותר.
 - מדיניות קריאה (Read Policy): מסך הלוגים בפאנל הניהול יציג כברירת מחדל את הנתונים מ-Redis (מהירים, אקטואליים); בעת חיפוש/סינון היסטורי (מעבר לטווח ה-TTL) המערכת תעבור לשליפה ישירה מ-Postgres.
 - עמידות (Fallback): אם Redis אינו זמין, המערכת תיפול (Fallback) אוטומטית לשליפה מ-Postgres, כדי שמסך הלוגים לא ייפול.
 - אבטחה: גישה למסך הלוגים מוגבלת אך ורק לתפקיד Admin (Authorization Middleware); הלוגים עצמם אינם ניתנים לעריכה או מחיקה (Read-Only) גם עבור Admin.
- שדות מרכזיים ברשומת לוג: log_id, user_id, action_type, timestamp, source_ip (בהתאם לטבלת AuditLog בסעיף 7). מבנה המפתח ב-Redis מומלץ: audit:recent (Sorted Set לפי timestamp) עם ה-value כ-JSON של הרשומה.

11. סיכום ומשימות פתוחות

- גיבוש Wireframes מלאים למסכים שפורטו בסעיף 6.
- קבלת החלטה סופית על תצורת ה-Free Tier המדויקת ב-Redis, Postgres+pgVector, Cloud Run) GCP ושירותים נלווים).
- החלטה סופית על שילוב Neo4j/Weaviate/Redis בשלב ה-POC לעומת שלבים מאוחרים יותר.
- הרחבת תרשים ה-ERD לרמת שדות מלאה (טיפוסי נתונים, אילוצים, אינדקסים) לקראת מימוש בפועל.
- הגדרת תהליכי טיפול בשגיאות ומקרי קצה: זיהוי שגוי של מזון, חוסר התאמה בהמלצה, כשל בגישה למאגרי מידע, כשל בשכבת ה-Cache.
- בחינת הרגולציה החלה על מידע בריאותי/תזונתי רגיש, בשיתוף גורמי אבטחת מידע.